

Universidad Siglo 21

Ingeniería en Software

Seminario Final

Módulo 2: Resultados

Trabajo Final de Graduación: Trabajo de Investigación

Tema de investigación:

**Interfaces cerebro-computadora bajo condiciones adversas: del
software a la decodificación**

Alumno: Agustín Tamagusuku

Legajo: SOF02612

Profesor tutor: Luis Miguel Nuñez

Fecha de entrega: 28 de septiembre de 2026

Introducción

Una interfaz cerebro-computadora (BCI, por su sigla en inglés) es un sistema que mide la actividad del sistema nervioso central y la convierte en una salida artificial, por ejemplo un texto en pantalla o el movimiento de un cursor, que reemplaza o complementa una salida natural del cuerpo (Wolpaw et al., 2002). Tiene aplicaciones en investigación con personas sanas y en neurorrehabilitación, pero uno de sus principales usos clínicos, y el que enmarca este trabajo, es la comunicación asistiva para personas con parálisis severa que conservan la capacidad de pensar pero no la de moverse o hablar. Los resultados recientes muestran hasta dónde llegó el campo: Willett et al. (2023) reportaron una neuroprótesis del habla, un dispositivo implantado que reemplaza la función de hablar, que permitió a una participante con esclerosis lateral amiotrófica, una enfermedad neurodegenerativa que paraliza progresivamente los músculos, producir texto a 62 palabras por minuto a partir de su actividad cortical, Metzger et al. (2023) describieron un sistema que decodificó habla y expresiones faciales para controlar un avatar digital, y Card et al. (2024) mostraron que un sistema comparable pudo calibrarse en pocas sesiones con tasas de error de palabra inferiores al 3 %.

Para entender qué se estudia en este trabajo conviene seguir el recorrido de la señal. Nicolas-Alonso y Gomez-Gil (2012) describieron las etapas de toda BCI: la adquisición, en la que electrodos captan la actividad eléctrica del cerebro, en la variante no invasiva mediante electroencefalografía (EEG) sobre el cuero cabelludo, donde cada electrodo es un canal y cada valor de voltaje que se registra, cientos de veces por segundo, es una muestra; el transporte, que lleva esa señal desde el equipo de registro hasta los programas que la procesan, muchas veces entre procesos o máquinas distintas; el preprocesamiento, que filtra la señal y la segmenta en ventanas, tramos de duración fija

que se analizan de a uno; la decodificación, en la que un modelo reconoce en cada ventana cuál de un conjunto de intenciones está ejecutando la persona; y la decisión, que convierte esa intención en una acción (Figura 1). El modelo de decodificación aprende antes de usarse, a partir de registros en los que se sabe qué imaginó la persona en cada repetición de la tarea: cada tramo de señal va acompañado de su respuesta correcta, llamada etiqueta (por ejemplo, «mano izquierda»), y el modelo ajusta sus parámetros hasta asociar los patrones de la señal con esas etiquetas; una vez entrenado, se aplica a señal nueva, cuya etiqueta desconoce. Un ejemplo sencillo es la imaginiería motora (*motor imagery*), que consiste en imaginar un movimiento sin ejecutarlo: la persona imagina mover la mano izquierda o la derecha, ese acto produce patrones distinguibles en el EEG sobre la corteza motora, la región del cerebro que controla el movimiento voluntario, de cada hemisferio, y el decodificador clasifica cada ventana como izquierda o derecha para mover un cursor. Todo ese recorrido ocurre en tiempo real, muchas veces por segundo, y se ejecuta sobre una cadena de software que en la investigación es, en su mayor parte, de código abierto y compartida entre laboratorios. Esa cadena de software, y no el implante ni el modelo, es el objeto de esta investigación; la calidad de las decisiones que produce el decodificador es lo que el usuario percibe como funcionamiento o falla.

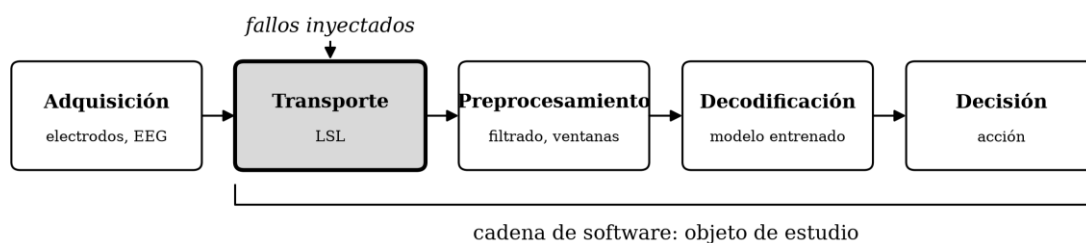


Figura 1. Etapas de una interfaz cerebro-computadora y objeto de estudio. Elaboración propia a partir de las etapas descritas por Nicolas-Alonso y Gomez-Gil (2012). En gris, la etapa sobre la que se inyectan los fallos; el corchete abarca la cadena de software que se estudia.

Para la persona que usa el sistema, el desempeño del decodificador no es un porcentaje sino una cantidad de comunicación por unidad de tiempo. Wolpaw et al. (2002) reportaron tasas máximas de transferencia de información de 10 a 25 bits por minuto para las BCI de su época y describieron que esa tasa depende de la exactitud de cada selección, del número de opciones disponibles y del tiempo que insume cada decisión; una caída de exactitud se paga, además, con las correcciones que el usuario debe hacer para deshacer las selecciones erróneas. Los sistemas recientes alcanzaron velocidades de comunicación muy superiores en tareas como la decodificación del habla, que se reportan en palabras por minuto y no son directamente comparables con aquellas tasas en bits, pero la relación de fondo se mantiene: una pérdida de exactitud en la decodificación se traduce, en una medida que depende de la tarea y del protocolo, en tiempo y esfuerzo adicionales para una persona con movilidad muy limitada. Por eso este trabajo mide el efecto de las fallas de software en el desempeño del decodificador y no solo en el estado técnico del sistema.

La decodificación de imaginería motora tiene métodos y recursos consolidados. Blankertz et al. (2008) sistematizaron los patrones espaciales comunes (CSP, por su sigla en inglés), una técnica que combina los canales del EEG para resaltar las diferencias entre dos clases, habitualmente seguida de un análisis discriminante lineal (LDA), un clasificador que separa las clases con una frontera lineal; Lotte et al. (2018) revisaron diez años de algoritmos de clasificación y concluyeron que esas combinaciones lineales seguían siendo referencias competitivas, y Roy et al. (2019) relevaban el uso creciente de *deep learning* (aprendizaje profundo) advirtiendo sobre la dificultad de comparar resultados entre estudios. Para hacerlos comparables existen recursos públicos: Tangermann et al. (2012) documentaron los datos de la competencia BCI Competition IV, entre ellos el conjunto 2a de imaginería motora en nueve sujetos; Jayaram y Barachant

(2018) desarrollaron MOABB, que estandariza la carga de esos conjuntos y la evaluación de algoritmos, y Gramfort et al. (2013) aportaron MNE-Python como biblioteca de análisis de EEG.

El software que ejecuta el sistema completo también se comparte. Schalk et al. (2004) presentaron BCI2000 y Renard et al. (2010) OpenViBE; más recientemente el ecosistema se desplazó hacia Python con BciPy (Memmott et al., 2021), MEDUSA (Santamaría-Vázquez et al., 2023), Timeflux (Clisson et al., 2019) y el marco con el ser humano en el lazo de Gemborn Nilsson et al. (2023), y Ali et al. (2024) publicaron BRAND para experimentos en lazo cerrado, en los que la salida del sistema realimenta al usuario en tiempo real, sobre datos intracorticales, es decir, registrados con electrodos implantados dentro de la corteza cerebral. Estos ocho artículos describen la arquitectura y las capacidades de cada herramienta, y varios reportan latencias en condiciones nominales; ninguno reporta el desempeño de un decodificador bajo una condición de falla del software. Estas herramientas son marcos de aplicación: sistemas completos que resuelven la presentación de estímulos, el procesamiento y la decodificación. Para recibir la señal del equipo de registro, la mayoría se apoya en una capa de transporte común, que se describe a continuación; BRAND es la excepción, porque utiliza para ese fin la base de datos en memoria Redis (Ali et al., 2024).

En la capa de transporte hay un estándar de facto: Lab Streaming Layer (LSL), una biblioteca de código abierto que permite que distintos programas, en una misma máquina o en varias conectadas en red, publiquen y reciban flujos de señal en tiempo real con sus marcas de tiempo sincronizadas, de modo que el equipo que registra el EEG, el que lo procesa y el que presenta los estímulos puedan trabajar como un solo sistema. El problema que resuelve es la sincronización: un experimento suele combinar varios

equipos, como el EEG, un seguidor de la mirada o el programa que presenta las consignas, cada uno con su propio reloj, y analizar sus datos en conjunto exige saber con precisión de milisegundos qué ocurrió en cada uno al mismo tiempo; LSL lo resuelve en software, sin requerir un reloj de hardware común. Kothe et al. (2025) la describieron y reportaron, según su propio relevamiento, más de 2.300 menciones en artículos científicos, más de 150 clases de dispositivos compatibles y la integración en BCI2000, OpenViBE, MNE-Python, Timeflux y MEDUSA. La documentación del proyecto enumera integraciones con equipos de EEG de fabricantes como Brain Products, g.tec, ANT Neuro, BioSemi, Emotiv y OpenBCI, con seguidores de la mirada como Tobii y SR Research, con sistemas de captura de movimiento como Vicon y Qualisys, y con programas de presentación de estímulos como PsychoPy, E-Prime y Unity (Lab Streaming Layer, s. f.-b). Esa ubicuidad es la que hace relevante su comportamiento ante fallas: una propiedad no medida de LSL es una incertidumbre compartida por todos los experimentos que lo usan.

El artículo describe con detalle la arquitectura: los datos viajan por una red local convencional, con UDP, un protocolo sin confirmación de entrega, para descubrir los flujos disponibles, y una conexión TCP, que garantiza la entrega y el orden de los datos, para transmitirlos entre el productor, el programa que emite el flujo, y el consumidor, el que lo recibe; los relojes de las máquinas se sincronizan con un algoritmo derivado del protocolo de tiempo de red (NTP), el mismo principio con el que las computadoras ajustan su hora, con una nueva estimación del desfase cada cinco segundos; los búferes (memorias intermedias que retienen datos mientras el consumidor no puede procesarlos) tienen capacidad configurable, de modo que si un consumidor deja de recibir momentáneamente los datos quedan retenidos hasta que pueda tomarlos; y si un productor desaparece, el consumidor lo busca periódicamente y se reconecta cuando reaparece,

incluso en otra máquina con otra dirección. Los autores afirman que la biblioteca se somete periódicamente a pruebas de estrés con cientos de flujos y desconexiones, apagados y reconexiones aleatorios, y reportan como resultado de campo un *jitter* con desviación estándar de aproximadamente 0,5 ms entre flujos de EEG y electromiografía, el registro de la actividad muscular. Lo que ese artículo no reporta es lo que este trabajo busca medir: qué proporción de muestras se pierde y cuánto tarda la recuperación cuando esos mecanismos actúan, y qué efecto tiene ese comportamiento sobre las etapas posteriores de la cadena, en particular sobre la decodificación. En ese sentido, las propiedades de tolerancia a fallos de LSL son, hasta donde alcanza esta revisión, autorreportadas y cualitativas.

Para dimensionar la actividad de esos proyectos se consultaron, el 16 de agosto de 2026, dos indicadores públicos de sus repositorios en GitHub, la plataforma donde se aloja y versiona su código: la cantidad de usuarios que los marcaron como favoritos (*estrellas*), medida habitual aunque indirecta de popularidad, y la fecha de su última modificación publicada. LSL registraba cambios recientes y una comunidad amplia, en consonancia con su rol de estándar de facto; BciPy y MEDUSA registraban cambios recientes pero comunidades reducidas, de entre veinte y algo más de ciento cincuenta estrellas; Timeflux no registraba cambios desde diciembre de 2024; y BRAND se presentaba como infraestructura de un consorcio académico y no como estándar. Esos indicadores son indirectos y no miden la calidad del software ni la existencia de verificación, pero el panorama que muestran, una capa de transporte concentrada en una herramienta y una capa de aplicación fragmentada entre proyectos pequeños y de actividad desigual, refuerza el interés de contar con un protocolo de evaluación

independiente y reutilizable, y sugiere que una debilidad no medida en la capa de transporte alcanzaría a la vez a todos los marcos que se apoyan en ella.

Una cadena en tiempo real impone restricciones que el análisis fuera de línea no tiene. Gemborn Nilsson et al. (2023) diseñaron su marco con cuatro módulos, un coordinador central, un módulo de cálculo construido sobre Timeflux que preprocesa, entrena y clasifica, y dos interfaces gráficas, comunicados por LSL para los datos y por *websockets*, un canal de comunicación bidireccional con el navegador, para la presentación de estímulos; al caracterizar el sistema documentaron que los propios dispositivos periféricos introducen retrasos considerables, entre 20 y 40 ms con 5 ms de *jitter* en un casco de EEG inalámbrico y entre 9 y 117 ms en los monitores, y señalaron como limitación de fondo que una decodificación más avanzada consume un tiempo que obliga a analizar los datos fuera de línea.

Wilson et al. (2010) habían definido la latencia de un sistema BCI, el tiempo que tarda un cambio en la entrada en producir un cambio causalmente relacionado en la salida, como la suma de las contribuciones de la conversión analógica-digital, el procesamiento y la salida, más el *jitter* entre bloques; midieron BCI2000 y una implementación en MATLAB sobre cuatro computadoras y dos sistemas operativos, encontraron latencias de procesamiento de hasta 50 ms, salidas de video de 5 a 20 ms según el monitor y el sistema operativo, y un rango de 8 a 141 ms en MATLAB, y concluyeron que la elección del hardware y del sistema operativo tiene un efecto significativo sobre la temporalidad del experimento. Ambos trabajos miden sistemas que funcionan correctamente; ninguno introduce una falla para observar cómo la cadena responde.

De ahí se toman las tres propiedades del flujo de datos que este trabajo mide: la integridad, que todas las muestras emitidas lleguen; la temporalidad, que lleguen cuando

corresponde y a intervalos regulares; y la disponibilidad, que el flujo no se interrumpa. La pérdida de muestras (*sample loss*) compromete la primera, el retraso y el *jitter* (llegada irregular de los datos) la segunda, y la desconexión la tercera. La literatura sobre robustez de la decodificación, en cambio, perturbó la señal y no el software: Urigüen y Garcia-Zapirain (2015) revisaron la eliminación de artefactos del EEG, es decir, de contaminaciones no cerebrales de la señal como parpadeos o movimientos musculares, y Meng et al. (2018) estudiaron cómo la reducción del número de electrodos afectó el desempeño de una BCI en línea; en ambos casos el transporte se tomó como dado.

Estas fallas no son situaciones excepcionales, sino parte del funcionamiento cotidiano de una BCI. Los cascos de EEG se conectan a la computadora muchas veces por enlaces inalámbricos, que pueden perder la señal por unos instantes; LSL transporta los datos por red, incluso entre computadoras distintas (Kothe et al., 2025), y una red puede congestionarse o cortarse y restablecerse; y el programa que procesa la señal comparte la máquina con otros procesos, de modo que puede demorarse si la computadora está ocupada. Cualquiera de estas situaciones altera el flujo de datos que recibe el pipeline. Ocurren, sin embargo, en capas distintas del sistema. La pérdida de alcance de un dispositivo inalámbrico afecta a la adquisición, antes de que la señal entre en LSL, y puede eliminar muestras. La congestión de la red afecta al transporte, donde el protocolo TCP que LSL utiliza retransmite lo que se pierde, de modo que el efecto no es una muestra faltante sino más latencia y más variabilidad. Un consumidor demorado afecta al procesamiento: las muestras se acumulan en un búfer de capacidad finita que, según la documentación de LSL, descarta las más antiguas cuando se llena (Lab Streaming Layer, s. f.-a). Todas esas situaciones se manifiestan en el mismo lugar, la interfaz por la que el pipeline recibe el flujo, como muestras que faltan, que llegan tarde o a intervalos

irregulares, o que dejan de llegar por un tiempo (Figura 2). Los cuatro modelos de fallo de esta investigación, pérdida de muestras, *jitter*, retraso y desconexión, se definen sobre esa interfaz: son perturbaciones observables del flujo que entra al pipeline, con orígenes posibles en más de una capa, y no se presentan como fallas propias del protocolo LSL.

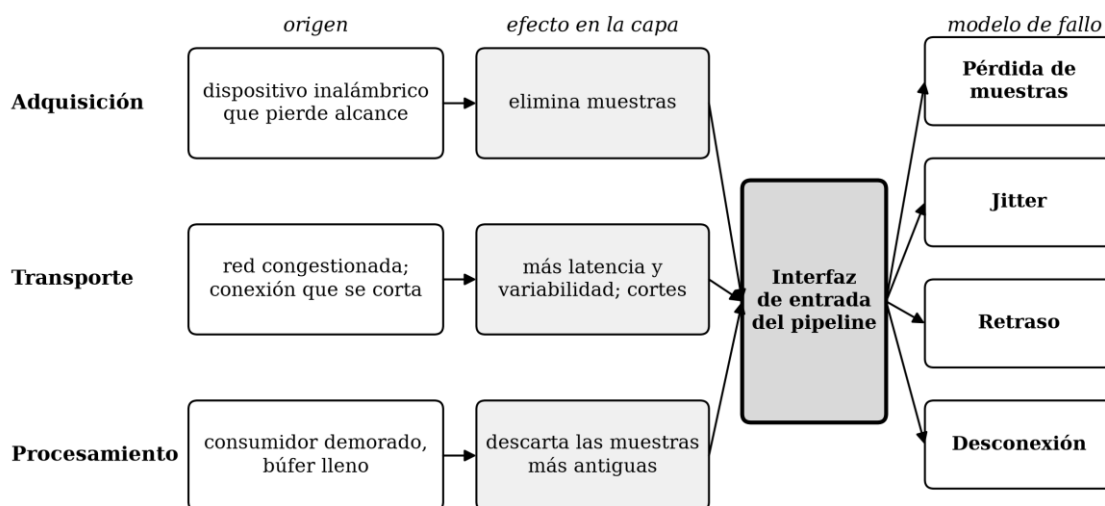


Figura 2. Orígenes de las perturbaciones en capas distintas y su manifestación común en la interfaz de entrada del pipeline. Elaboración propia. Los orígenes y efectos por capa son los descritos en el texto (Kothe et al., 2025; Lab Streaming Layer, s. f.-a); los cuatro modelos de fallo se definen sobre la interfaz de entrada, no sobre una capa en particular.

Fuera del dominio BCI, la ingeniería de software tiene un cuerpo de conocimiento maduro para este tipo de pregunta. Avizienis et al. (2004) fijaron la taxonomía de la computación confiable (*dependable computing*), con la distinción entre falla, error y fallo de servicio, describieron la tolerancia a fallos como detección de errores más recuperación, e incluyeron los fallos en los que el sistema no señala la degradación de su servicio (*silent failures*). Natella et al. (2016) sistematizaron la inyección de fallos de software como el método que anticipa los escenarios de peor caso provocados por software defectuoso mediante la introducción deliberada de fallas, revisaron cómo evolucionaron sus técnicas en torno a tres criterios, la representatividad de las fallas

inyectadas respecto de las reales, la eficiencia de las campañas y la usabilidad de las herramientas, y documentaron sus aplicaciones a la evaluación de sistemas tolerantes a fallos. Basiri et al. (2016) describieron cómo Netflix llevó esa idea a sistemas distribuidos en producción bajo el nombre de *chaos engineering* (ingeniería del caos), a partir de un servicio interno que apagaba al azar instancias de sus propios servidores para obligar a diseñar software que tolerara esas pérdidas, y la formalizaron en cuatro principios: formular una hipótesis sobre el comportamiento estable del sistema, variar eventos del mundo real, correr los experimentos en el entorno real y automatizarlos para que corran de forma continua. Los dos primeros principios son los que esta investigación adopta: una hipótesis sobre el estado estable, que aquí es el desempeño del pipeline sin fallos, y perturbaciones que reproducen eventos ordinarios de operación.

Los sistemas de procesamiento de flujos (*stream processing*), que como una BCI operan sobre secuencias continuas de eventos con restricciones temporales, fueron objeto de evaluaciones de este tipo. Karimov et al. (2018) compararon plataformas distribuidas con cargas de trabajo controladas en rendimiento y latencia, y Vogel et al. (2024) llevaron la comparación al terreno de la recuperación ante fallos: sobre Flink, Kafka Streams y Spark Structured Streaming desplegados en Kubernetes, inyectaron con la herramienta Chaos Mesh la eliminación y la caída de instancias a intervalos regulares, y midieron el rendimiento, la latencia de los eventos en percentiles, el retraso acumulado del consumidor y el tiempo de recuperación, detectado automáticamente a partir de la media móvil y la desviación de las series. Encontraron que Flink fue el más estable y el de recuperación más rápida, que Kafka Streams mostró inestabilidad después de las fallas por su estrategia de rebalanceo, la forma en que redistribuye el trabajo entre las instancias

que quedan, con casos en los que no se recuperó por completo antes de la falla siguiente, y que Spark se recuperó de manera consistente pero con mayor latencia.

Ese trabajo ofrece el molde metodológico que esta investigación adapta: familias de fallo, niveles de severidad, condición de referencia sin fallo, métricas de sistema medidas en el tiempo y un criterio explícito de recuperación. A diferencia de esos sistemas, una BCI tiene al final de la cadena un decodificador cuyas decisiones pueden compararse con lo que la persona realmente imaginó. Esa diferencia permite medir no solo si el sistema se recupera de una falla, sino si sigue decidiendo bien, y es la pieza que este trabajo agrega al molde metodológico.

Dicho de manera sencilla: ninguno de esos trabajos altera deliberadamente el transporte de la señal y observa, en el mismo experimento, dos cosas a la vez: si el sistema registra que algo anda mal y si el decodificador sigue acertando. De esos tres grupos de fuentes, plataformas de BCI, robustez de la decodificación e inyección de fallos en sistemas de propósito general, ninguno combina una falla inyectada en el transporte de señal con una medición simultánea del estado del sistema y del desempeño de un decodificador contra etiquetas conocidas. Para acotar esa afirmación se realizó en septiembre de 2026 una búsqueda en Google Scholar, Semantic Scholar, arXiv, PubMed e IEEE Xplore con términos que combinaron *fault injection*, *packet loss*, *jitter* y *Lab Streaming Layer* con *BCI* y *EEG*. Los trabajos más próximos cubren una parte de esa combinación: Zheng et al. (2025) simuló cuatro modelos de pérdida de paquetes sobre registros intracorticales de macaco y midieron su efecto sobre un decodificador del movimiento del brazo, sin medir el estado del sistema, y Kothe et al. (2025) sometieron la propia capa de transporte a pruebas de estrés, sin medir su efecto sobre la decodificación. No se encontraron, en esas bases y con esos términos, trabajos que

combinen ambas mediciones. Estudiarla no requiere hardware: MOABB provee la carga de los conjuntos públicos y el ecosistema de MNE-Python incluye componentes que reproducen una grabación como flujo LSL en tiempo real (Gramfort et al., 2013; Kothe et al., 2025), de modo que el mismo software que un laboratorio usaría con un amplificador y una persona puede ejecutarse con una fuente reproducible y variar solo aquello que se quiere estudiar.

El problema que aborda esta investigación es el desconocimiento sobre qué le ocurre a la decodificación de una BCI cuando el software que transporta la señal falla. No se sabe con qué magnitud la pérdida de muestras, el *jitter*, el retraso y la desconexión alteran la integridad, la temporalidad y la disponibilidad del flujo que recibe el decodificador, ni a partir de qué severidad esas alteraciones se traducen en una caída del desempeño de clasificación. Tampoco se sabe si hay condiciones en las que el proceso sigue activo, el flujo conectado y sin excepciones mientras la calidad de las decisiones cae (Figura 3), ni si la telemetría del propio sistema, las mediciones que registra sobre su propio estado, alcanza para detectar esa caída. Este último punto es el que más pesa cuando el usuario depende del sistema para comunicarse: una falla que el sistema informa puede gestionarse desde afuera; una que no informa solo se descubre por sus consecuencias.

Función del decodificador	Estado de la infraestructura	
	Operativa	No operativa
No degradada	Concordancia funciona y decide bien	Divergencia: interrupción no operativa, decide bien
Degradada	Divergencia: falla silenciosa operativa, decide mal	Concordancia falla visible y decide mal

Figura 3. Combinaciones posibles entre el estado de la infraestructura y la función del decodificador. Elaboración propia. Las dos celdas de divergencia son las que aborda el segundo interrogante; en gris, la falla silenciosa (*silent failure*), en la que el sistema no señala la degradación de su servicio (Avizienis et al., 2004).

La pregunta central es cómo se propagan fallos controlados de la capa de transporte de señal hacia las propiedades operativas y el desempeño funcional de un pipeline BCI en tiempo real, y en qué medida la telemetría del sistema permite detectar esa degradación. De ella se desprenden tres interrogantes, que se corresponden con las tres comparaciones del diseño experimental: ¿cómo afectan distintos tipos y severidades de fallo a la integridad de los datos, la disponibilidad, la latencia y el desempeño de decodificación del pipeline?; ¿existen condiciones de fallo bajo las cuales el estado observable de la infraestructura y el desempeño del decodificador divergen?; y ¿puede un modelo de aprendizaje automático, entrenado para reconocer la degradación a partir de varias mediciones del sistema a la vez (la telemetría: muestras recibidas, latencia, huecos, entre otras), detectarla mejor que un detector por umbrales, que dispara una alarma cuando una sola de esas mediciones supera un límite fijo? Las tres admiten un resultado

negativo: si la infraestructura absorbe los fallos, se documenta esa tolerancia; si no hay divergencia, ese resultado vale; y si un detector por umbrales rinde igual o mejor que el modelo, la complejidad adicional no se justifica. Asumir lo contrario de antemano habría condicionado el experimento hacia el resultado esperado.

La relevancia del problema proviene del lugar del software abierto en el campo: cuando una herramienta es gratuita, ubicua y la base sobre la que se validan y replican los resultados, una propiedad no medida de esa herramienta es una incertidumbre que arrastran todos los trabajos que corren sobre ella, y en una BCI del otro lado hay una persona que con frecuencia no dispone de otro canal para corregir una decisión errónea. Las BCI son una de las fronteras de la transformación digital en salud, y que su infraestructura sea confiable es condición para que lo construido sobre ella sea reproducible.

El planteo es conservador: somete al software a las condiciones que sus autores afirman tolerar, con datos públicos y sin hardware de adquisición, unidades de procesamiento gráfico ni acceso a pacientes. Toma de Vogel et al. (2024) la estructura del protocolo, de Natella et al. (2016) los criterios de diseño y de Basiri et al. (2016) la idea de verificar lo declarado; se aparta de ellos en el objeto, un pipeline BCI, y en la variable de respuesta, que incluye el desempeño de un decodificador. Sigue la tradición de la experimentación controlada en ingeniería de software (Wohlin et al., 2012), en la que el artefacto construido sirve para medir y no es el resultado. Lo que se espera obtener es un protocolo reproducible con su banco de pruebas abierto, las curvas que relacionen cada condición de falla con su efecto sobre la infraestructura y la decodificación, y una comparación entre un mecanismo de alerta por umbrales y un modelo basado en telemetría.

Conviene dejar claro desde ahora qué no pretende el estudio. El enfoque Software-in-the-Loop reproduce el comportamiento del software real con señal grabada, pero no reproduce los electrodos, los amplificadores, la interferencia electromagnética ni la adaptación de la persona a un sistema que le devuelve retroalimentación. Las conclusiones se refieren, por lo tanto, al comportamiento del pipeline de software bajo las condiciones estudiadas y no a la resiliencia de una BCI clínica completa; extender el protocolo a hardware en el lazo queda como trabajo futuro.

El objetivo general de esta investigación es caracterizar empíricamente cómo se propagan fallos controlados de la capa de transporte de señal, en particular la pérdida de muestras, el *jitter*, el retraso y la desconexión, hacia las propiedades operativas y el desempeño de decodificación de un pipeline BCI en tiempo real basado en software de código abierto, y evaluar en qué medida la telemetría del sistema permite detectar esa degradación.

De ese objetivo general se derivan los siguientes objetivos específicos. Construir un banco de experimentación Software-in-the-Loop, es decir, el sistema de software real alimentado con señal grabada en lugar de una persona, que reproduzca señal electroencefalográfica pública como flujo en tiempo real sobre Lab Streaming Layer, con inyectores de fallos parametrizables, un decodificador de referencia y un recolector de telemetría. Cuantificar el efecto de cada tipo y severidad de fallo sobre la integridad, la temporalidad y la disponibilidad del flujo de datos, en comparación con la condición de referencia sin fallos. Medir el efecto de esos mismos fallos sobre el desempeño del decodificador de referencia y describir las curvas de degradación funcional resultantes. Identificar las condiciones de fallo bajo las cuales el estado observable de la infraestructura y el desempeño funcional del decodificador divergen. Comparar la

capacidad de un detector basado en umbrales y la de un modelo de clasificación entrenado con telemetría multivariable para detectar la degradación funcional. Documentar el protocolo, los inyectores y los datos generados de manera que el estudio pueda replicarse y extenderse a otros pipelines.

Métodos

En síntesis, el estudio somete un pipeline BCI de código abierto a fallos controlados en su capa de transporte y mide, ensayo por ensayo, qué le ocurre al flujo de datos y al decodificador. La señal de nueve sujetos de un conjunto público se reproduce en tiempo real como si proviniera de un equipo de registro; un inyector perturba esa transmisión con seis tipos de fallo en tres severidades, cuatro de distribución uniforme en el tiempo y dos sincronizados con el ensayo; el pipeline decodifica con un modelo entrenado previamente y congelado, y guarda el segmento de señal que recibió en cada ensayo, sobre el que se evalúa un segundo decodificador; y un recolector registra, segundo a segundo, el estado del flujo y del proceso. Las secciones que siguen detallan el diseño experimental, la fuente de datos, el instrumento y el análisis.

Diseño

La investigación adopta un enfoque cuantitativo, con alcance explicativo sobre una base descriptiva: manipula deliberadamente las condiciones de falla para observar su efecto sobre variables de respuesta medidas numéricamente (Hernández Sampieri et al., 2014). El diseño es experimental, de medidas repetidas, con condición de control. El objeto de estudio es un pipeline BCI único; las unidades experimentales son los nueve sujetos del conjunto de datos, cada uno con dos sesiones: la primera entrena el decodificador sin fallos y fuera de línea, y la segunda se reproduce a través del pipeline bajo la condición de referencia y bajo cada condición de falla. Las observaciones son los ensayos clasificados en cada reproducción, y las comparaciones se hacen dentro de cada sujeto y entre condiciones. Las variables independientes son el tipo de fallo, con seis niveles en dos familias (pérdida de muestras, *jitter*, retraso y desconexión, de distribución uniforme en el tiempo; y pérdida contigua en el ensayo y desconexión en el ensayo,

sincronizadas con la ventana de decodificación), y su severidad, con tres niveles por tipo: son las que el experimento manipula, porque la pregunta de investigación indaga el efecto de una falla en el transporte. Las variables dependientes, que se detallan más adelante, siguen ese efecto a lo largo de la cadena: primero sobre el flujo de datos, después sobre la función del decodificador y, por último, sobre la capacidad del sistema de advertirlo, en correspondencia con los tres interrogantes. Se mantienen constantes la señal, el modelo, su configuración, el preprocesamiento y la máquina, de modo que el efecto de cada fallo se estima como la diferencia entre la métrica obtenida con el fallo y la obtenida sin él en la misma corrida del mismo sujeto ($\Delta = Y \text{ con fallo} - Y \text{ de referencia}$), lo que cancela las diferencias propias de cada sujeto. Las ejecuciones se organizan por bloques de corrida y se lanzan de a doce en el orden del plan; como cada una corre en procesos propios con su propia semilla, el orden no introduce dependencia entre condiciones.

Cada ejecución reproduce una corrida completa de la sesión de evaluación, de 48 ensayos y del orden de seis minutos y medio en tiempo real. De las seis corridas de cada sujeto, la sexta se reservó para la prueba piloto, de modo que los datos con los que se calibraron severidades, umbral y tamaño de ventana no participan de las comparaciones. El diseño es por bloques: cada corrida se reproduce sin fallo y bajo cada condición de falla, con una realización del fallo fijada por una semilla aleatoria, el valor que determina la secuencia de números aleatorios y hace repetible la ejecución. Toda comparación se hace, así, entre la misma corrida con y sin fallo, lo que separa el efecto de la falla de las diferencias naturales de clasificación entre corridas; las corridas no son repeticiones idénticas sino bloques que permiten estimar si el efecto se sostiene con datos distintos.

La campaña se realizó en dos etapas, en un diseño secuencial. La primera reprodujo las corridas primera y segunda de cada sujeto bajo la referencia y las doce

condiciones de la familia uniforme (234 ejecuciones) y no mostró efecto funcional de ningún fallo; a la luz de ese resultado, y antes de la segunda etapa, se agregó la segunda familia de modelos, estructurados, con severidades ancladas a la misma escala del ensayo, y se extendió el diseño a las cinco corridas previstas. Las condiciones de la familia uniforme se especificaron antes de observar resultados; la familia estructurada se incorporó de manera exploratoria, a partir del resultado de la primera etapa, con severidades fijadas antes de ejecutarla, por lo que sus comparaciones se interpretan como exploratorias y no como confirmatorias. La campaña definitiva, cuyos resultados se reportan, comprende 855 ejecuciones: por cada uno de los nueve sujetos, cinco corridas reproducidas bajo la referencia y las dieciocho condiciones de fallo, unas 95 horas de reproducción en tiempo real que el ejecutor de experimentos automatiza y distribuye en doce ejecuciones simultáneas. El piloto se ejecutó sobre la sexta corrida de tres sujetos elegidos por su desempeño de referencia bajo, medio y alto.

Participantes

La investigación no involucra participantes humanos, por lo que no corresponde consentimiento informado. La población de referencia son los pipelines BCI construidos sobre el ecosistema de código abierto cuyo transporte es LSL; la muestra, no probabilística e intencional, es un pipeline de referencia compuesto por MNE-LSL para la reproducción de la señal, código propio para inyección de fallos y telemetría, y un decodificador CSP más LDA sobre MNE-Python y scikit-learn. Se prefirió esa composición a un marco de aplicación completo como BciPy o MEDUSA porque los marcos incorporan sus propias capas de búfer y manejo de errores, y con ellos no sería posible atribuir un efecto al transporte o al marco; una segunda implementación sobre

uno de esos marcos se contempla como validez externa, después de la campaña principal. Las conclusiones se formulan para el pipeline y las condiciones estudiadas.

La señal proviene del conjunto público BCI Competition IV 2a, BNCI2014_001 en MOABB: EEG de 22 canales, es decir, 22 electrodos, y 3 canales de electrooculografía, que registran el movimiento ocular para identificar artefactos, a 250 Hz, o sea 250 muestras por segundo y por canal, nueve sujetos, dos sesiones de seis corridas de 48 ensayos, doce por clase, donde cada ensayo es una repetición de la tarea y cada corrida un bloque de ensayos, en una tarea de imaginería motora de cuatro clases con ensayos de cuatro segundos (Tangermann et al., 2012). La decodificación se restringe a mano izquierda y mano derecha porque CSP fue formulado para problemas binarios (Blankertz et al., 2008) y sus extensiones multiclase agregan una variabilidad del clasificador ajena a la pregunta. Cada corrida se reproduce completa, con sus 48 ensayos y pausas, para conservar la carga y la estructura temporal originales; el pipeline clasifica solo los 24 ensayos de mano izquierda o derecha, que son los que ingresan al cálculo de desempeño. Se excluyen los ensayos marcados por los autores del conjunto como contaminados por artefactos.

Instrumentos

El instrumento principal es el banco de experimentación Software-in-the-Loop, publicado con licencia abierta, con seis componentes (Figura 4): un servicio de reproducción que emite la señal como flujo LSL a su tasa original; un inyector de fallos entre fuente y consumidor; el pipeline bajo prueba, que segmenta ventanas, preprocesa, extrae características con CSP y clasifica con LDA, combinación elegida por ser el método de referencia sobre este conjunto (Tangermann et al., 2012), seguir siendo competitiva (Lotte et al., 2018), ser determinista y tener costo compatible con el tiempo

real, y que además conserva el segmento de señal recibido en cada ensayo para evaluar sobre él un segundo decodificador, la red convolucional compacta EEGNet (Lawhern et al., 2018), como análisis de robustez con una segunda familia de decodificador; un recolector de telemetría que registra por ventana el estado del flujo y del proceso; dos detectores de salud, uno por umbrales y otro basado en un modelo de clasificación; y un ejecutor que orquesta las campañas, fija semillas y persiste resultados junto con las versiones de software y hardware.

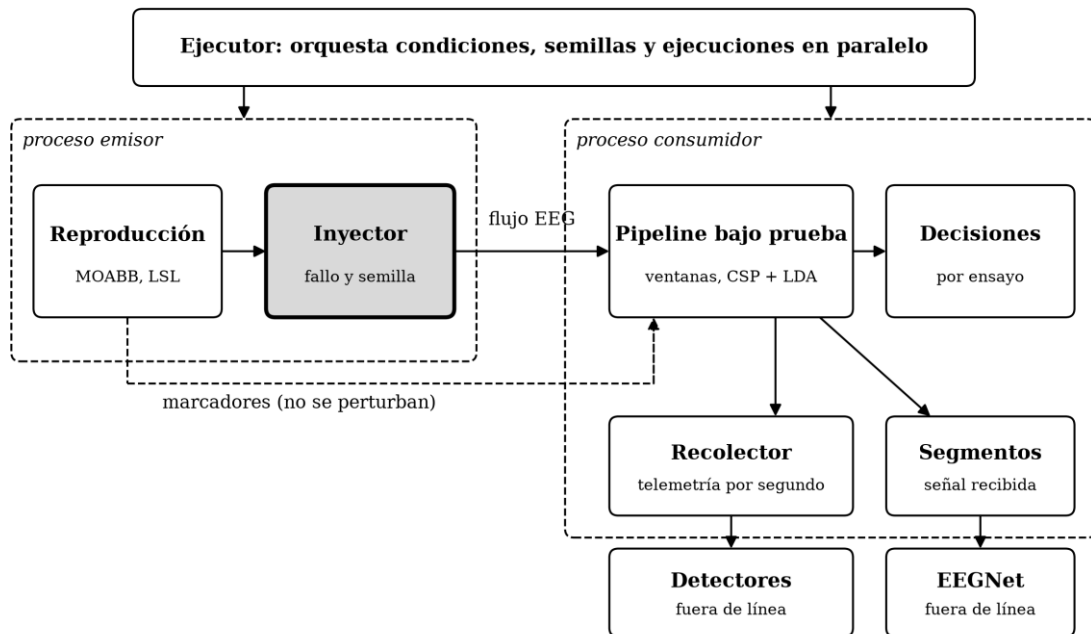


Figura 4. Componentes del banco de experimentación Software-in-the-Loop y flujo de datos entre ellos. Elaboración propia. En gris, el inyector de fallos. Los recuadros discontinuos delimitan los dos procesos del sistema; el flujo de marcadores no se perturba. Los detectores y EEGNet se evalúan fuera de línea sobre lo registrado.

El banco funciona como un simulador: el software real de una BCI recibe, en lugar de la señal de una persona conectada, una grabación reproducida con el mismo ritmo con que la produciría un equipo de registro, de modo que el sistema no distingue entre ambas y lo único que varía entre ejecuciones es el fallo que se inyecta. Los párrafos que siguen recorren el camino de la señal.

El servicio de reproducción emite la señal en bloques de diez muestras, uno cada 40 ms, en el instante en que un equipo real los habría registrado y con la marca de tiempo nominal de cada muestra; para no depender de la resolución del temporizador del sistema operativo, espera cada instante con una pausa del sistema seguida de una espera activa durante los últimos 2 ms. Por un segundo flujo LSL, que nunca se perturba, envía los marcadores que indican el inicio de cada ensayo, como lo haría el programa que presenta las consignas a la persona. El inyector actúa sobre el flujo de señal antes de que salga del proceso emisor.

El pipeline bajo prueba corre en otro proceso. Para cada ensayo toma los cuatro segundos de señal en los que la persona imagina el movimiento, la ventana de 2 a 6 s desde el marcador, con un segundo adicional a cada lado para filtrar entre 8 y 30 Hz sin efecto de borde, y la clasifica con seis componentes CSP y LDA; por eso la decisión se emite alrededor de un segundo después del fin del ensayo. Si al cerrarse la ventana llegó menos de medio segundo de señal (125 muestras) dentro de ella, o si los datos no llegan dentro de los tres segundos siguientes, la decisión se registra como inválida (Figura 5).

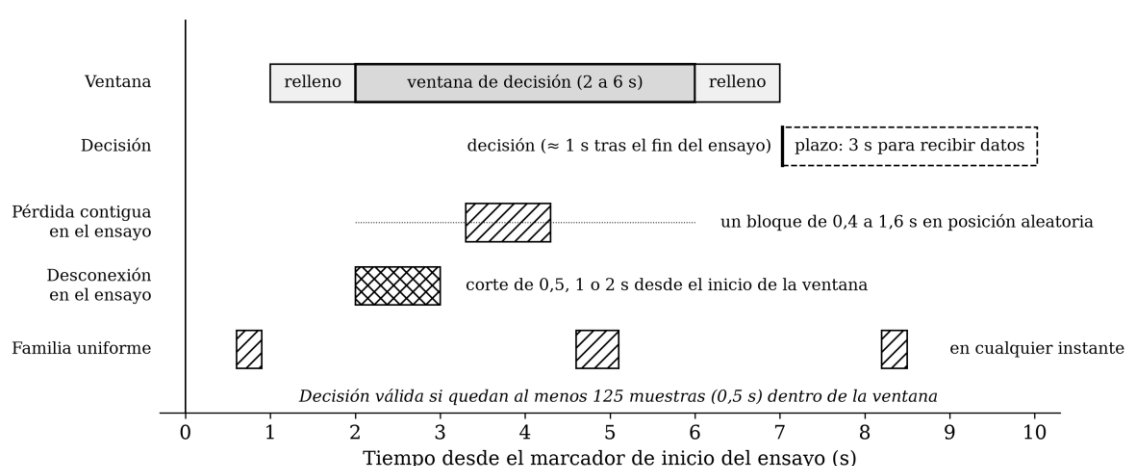


Figura 5. Línea de tiempo de un ensayo: ventana de decodificación, decisión y ubicación de los fallos de cada familia. Elaboración propia. Tiempos respecto del marcador de inicio del ensayo. Las posiciones de los fallos son ilustrativas: en la familia estructurada el fallo cae dentro de la ventana de cada ensayo; en la uniforme, en instantes aleatorios de la corrida.

Durante la ejecución se guardan dos registros: el segmento de señal de cada ensayo tal como llegó, con sus huecos y sus marcas de tiempo, y la telemetría del sistema, que el recolector resume por segundo nominal en las muestras esperadas y recibidas, los huecos, la latencia entre marca de tiempo y llegada, el intervalo entre bloques recibidos, las predicciones emitidas y las excepciones del proceso. Los dos detectores de salud se evalúan después, fuera de línea, sobre esa telemetría, ventana a ventana.

El segundo decodificador, EEGNet, se entrena una vez por sujeto con la sesión de entrenamiento (300 épocas, parámetros de referencia del artículo original para 250 Hz) y se evalúa fuera de línea sobre los segmentos guardados. Como la red necesita siempre una entrada del mismo tamaño, los cuatro segundos de la ventana (mil muestras) se reconstruyen en su posición original y los lugares donde faltan muestras se completan con ceros. Esa entrada no es idéntica a la que recibe CSP y LDA en línea, que filtra la señal sin los huecos; por eso EEGNet se usa como análisis de robustez con una segunda familia de decodificador y no como una comparación pura entre arquitecturas. Para acotar esa diferencia se reaplicó CSP y LDA a los segmentos guardados y se compararon sus decisiones con las tomadas en línea (véase Resultados).

El banco se implementó en Python 3.12 sobre MNE-Python 1.13, MNE-LSL 1.14, MOABB 1.7 y scikit-learn 1.9. Piloto y campaña se ejecutaron en una máquina virtual dedicada (Amazon EC2 c6i.2xlarge: ocho procesadores virtuales, 16 GB de memoria, Ubuntu 24.04), elegida porque el temporizador de Windows, con una granularidad del orden de 15 ms, no permite inyectar un *jitter* de 10 ms; el efecto de ejecutar varias reproducciones simultáneas sobre las mediciones temporales se controló comparando la condición de referencia ejecutada sola con la ejecutada en paralelo.

Los modelos de fallo se definen, siguiendo a Avizienis et al. (2004), como fallas que se manifiestan en la interfaz de transporte del pipeline y que pueden o no propagarse como errores en su estado y como fallos de servicio en la salida del decodificador. La Tabla 1 resume, para cada modelo, dónde se aplica, con qué severidades, qué propiedad mide y qué situación real representa.

Tabla 1

Modelos de fallo de las dos familias: aplicación, severidades, propiedad medida y situación real que representan

Modelo	Dónde se aplica	Severidades	Qué mide	Situación real
Pérdida de muestras	Contenido del flujo: el inyector omite muestras al azar, de manera uniforme en el tiempo	1 %, 5 %, 10 % de las muestras	Tolerancia del pipeline a datos faltantes	Dispositivo que pierde muestras antes de LSL; búfer del consumidor que descarta las más antiguas al saturarse
<i>Jitter</i>	Instante de entrega de cada bloque: perturbación aleatoria	Desviación de 10, 50, 100 ms	Tolerancia a la irregularidad temporal	Congestión de red; retransmisiones de TCP; consumidor demorado
Retraso	Instante de entrega de cada bloque: desplazamiento constante	50, 100, 250 ms	Tolerancia a la latencia	Los mismos orígenes que el <i>jitter</i> , con efecto sostenido
Desconexión	Transporte real: el flujo de origen se cierra y se vuelve a crear, cinco veces por corrida en instantes aleatorios	0,5, 1, 3 s hasta la reconexión	Reconexión y búferes que LSL declara (único modelo que los ejercita directamente)	Corte del enlace; caída del proceso productor
Pérdida contigua en el ensayo	Contenido del flujo: un solo bloque de muestras consecutivas, dentro de la ventana de decodificación de cada ensayo, en posición aleatoria	10 %, 25 %, 40 % de la ventana (0,4, 1, 1,6 s)	Tolerancia a un hueco concentrado durante la decisión	Corte breve del enlace inalámbrico durante el ensayo (<i>dropout</i>)

Modelo	Dónde se aplica	Severidades	Qué mide	Situación real
Desconexión en el ensayo	Transporte real: el corte comienza al inicio de la ventana de decodificación de cada ensayo	0,5, 1, 2 s hasta la reconexión	Reconexión de LSL cuando el corte coincide con la decisión	Corte del enlace en el momento en que la persona ejecuta la tarea

Nota. Las cuatro primeras filas son la familia uniforme, de distribución aleatoria en el tiempo; las dos últimas, la familia estructurada, sincronizada con la ventana de decodificación de cada ensayo. Los rangos toman como referencia el ensayo de cuatro segundos a 250 Hz (mil muestras): una desconexión de 0,5 s elimina una octava parte de un ensayo y una de 3 s casi lo elimina; los retrasos van del orden de un bloque de transmisión a un cuarto de segundo; el *jitter* se ubica por debajo, en el orden y por encima de la duración de un bloque; la pérdida uniforme va de una fracción pequeña al equivalente a cien muestras por ensayo; la pérdida contigua y la desconexión en el ensayo van de una fracción de la ventana a casi la mitad.

Conviene subrayar la distinción que la tabla recoge: LSL transmite sobre TCP, de modo que una pérdida de paquetes de red no elimina muestras sino que las demora; por eso la pérdida de muestras representa lo que ocurre antes de LSL o en un búfer saturado, mientras que el *jitter* y el retraso son el síntoma propio de la red y del consumidor, y solo la desconexión pone a prueba los mecanismos de recuperación del propio LSL. Las severidades de la familia uniforme se fijaron tras la prueba piloto y las de la familia estructurada antes de ejecutar la segunda etapa; en ningún caso se modificaron después de observar sus propios resultados.

Las variables dependientes se agrupan en cinco familias. Integridad: relación entre muestras recibidas y esperadas, número y duración de huecos, pérdida total y en ráfagas. Temporalidad: latencia de extremo a extremo, media y desviación del intervalo entre llegadas, tiempo de recuperación. Disponibilidad: continuidad del flujo, reconexiones, proporción de ventanas descartadas. Desempeño funcional, para cada uno de los dos decodificadores: *balanced accuracy* (exactitud balanceada, el promedio de los aciertos obtenidos en cada clase por separado) por sujeto y condición como métrica principal, complementada con exactitud simple, confianza media y tasa de decisiones válidas. Se

prefiere a la exactitud simple porque los fallos invalidan ensayos de manera desigual entre clases: si de los ensayos válidos 10 fueron de mano izquierda y 4 de mano derecha, un decodificador que respondiera siempre «izquierda» acertaría 10 de 14 (71 % de exactitud simple) sin reconocer nunca la mano derecha, mientras que su exactitud balanceada sería de $(100 \% + 0 \%) / 2 = 50 \%$, el nivel del azar. Observabilidad: para cada detector, tasa de verdaderos positivos, tasa de falsos positivos, retardo de detección (*detection delay*) y precisión promedio (*average precision*, el área bajo la curva que relaciona precisión y sensibilidad para todos los umbrales de alarma posibles), que se compara con la prevalencia de la degradación, es decir, con la precisión que obtendría un detector que disparara al azar. Se define como estado operativo la conjunción de proceso activo, flujo conectado, ausencia de excepciones y llegada continua de muestras y predicciones; y como degradación funcional, una caída de la *balanced accuracy* que supere el ruido de la referencia del mismo sujeto. Para que ambas magnitudes estén en la misma escala, la *balanced accuracy* se calcula siempre sobre una ventana móvil (*rolling window*) de ensayos consecutivos: la distribución de referencia se obtiene aplicando esa misma ventana a las corridas sin fallo del sujeto, y el umbral se fija a partir de su dispersión. El tamaño de la ventana y la regla del umbral se fijaron en la prueba piloto y permanecieron constantes en la campaña principal: ventana de ocho ensayos consecutivos, y umbral igual al mínimo de la *balanced accuracy* móvil observado en las corridas sin fallo del mismo sujeto, de modo que se considera degradación toda ventana peor que cualquiera de las observadas sin fallo. Se prefirió esa regla a un percentil o a un múltiplo de la desviación absoluta mediana porque estas marcan por construcción una fracción de las ventanas de referencia como degradadas, lo que en el piloto resultó indistinguible del ruido. El inicio

de la degradación es la primera ventana que cruza el umbral, y respecto de él se mide el retardo de detección.

Análisis de datos

El análisis es cuantitativo y se realiza en Python con las bibliotecas pandas, SciPy y scikit-learn, de manejo de datos, estadística y aprendizaje automático respectivamente. Se describen las variables por sujeto, tipo y severidad y se construyen curvas de degradación por variable. La unidad estadística es el sujeto. Las decisiones de un mismo sujeto no son independientes entre sí, porque provienen del mismo cerebro, la misma sesión y el mismo equipo; tratarlas como observaciones independientes inflaría artificialmente el tamaño de la muestra y haría significativa cualquier diferencia mínima (pseudorreplicación). Por eso, para cada sujeto y condición, las cinco corridas se resumen en un único valor, su mediana, y son esas nueve observaciones por condición las que ingresan a las pruebas; las tablas reportan la mediana de esos nueve valores. Se usa la mediana y no el promedio porque, con tan pocas observaciones, un valor atípico, como una corrida anómala, desplazaría el promedio sin representar lo típico.

Para el primer interrogante, cada condición de falla se compara con la de referencia mediante pruebas no paramétricas para muestras apareadas. Se eligen no paramétricas porque con nueve sujetos no es posible verificar que los datos sigan una distribución normal, supuesto de pruebas como la t de Student o el análisis de varianza, y porque la *balanced accuracy* calculada sobre 24 ensayos solo toma valores discretos. Para cada tipo de fallo, una prueba de Friedman, el equivalente no paramétrico del análisis de varianza de medidas repetidas, indica si alguna de las cuatro condiciones (la referencia y las tres severidades) difiere de las demás; una prueba de Wilcoxon para muestras apareadas compara además cada severidad con la referencia, sujeto por sujeto, y se

reportan ambos niveles. Como cada tipo de fallo implica tres comparaciones, y la probabilidad de obtener al menos un falso positivo por azar crece con el número de comparaciones, los valores p de Wilcoxon se ajustan con la corrección de Holm dentro de cada tipo de fallo. Con nueve sujetos, el menor valor p corregido alcanzable es de aproximadamente 0,012, que corresponde al caso en que los nueve cambian en la misma dirección. Junto a cada comparación se informa el tamaño de efecto r , el estadístico normal asociado al valor p dividido por la raíz del número de sujetos, que indica qué tan consistente fue el cambio entre sujetos: se aproxima a su máximo, cercano a 0,96 con nueve sujetos, cuando todos se movieron en el mismo sentido. Para el segundo, se cruzan ventana a ventana el estado operativo y la degradación funcional y se cuantifica la proporción de ventanas en que divergen, por condición.

Para el tercero, el detector por umbrales y el modelo se evalúan sobre las mismas ventanas etiquetadas por su estado funcional medido, con validación cruzada que deja fuera un sujeto completo en cada iteración, es decir, entrenando con ocho sujetos y probando con el restante, para evitar *data leakage* (filtración de información entre los datos de entrenamiento y los de prueba), y se comparan por precisión, sensibilidad, puntaje F1, precisión promedio y retardo de detección. No se evalúa la anticipación: como la degradación se etiqueta con una ventana móvil de ocho ensayos, su inicio registrado se retrasa respecto del fallo que la causa, y una alarma previa a ese inicio no podría distinguirse de una detección del fallo anterior a la etiqueta. Los modelos candidatos son la regresión logística, como referencia lineal e interpretable, y los algoritmos *random forest* (bosques aleatorios) y *gradient boosting* (aumento de gradiente), porque la hipótesis del tercer interrogante es que la degradación se reconoce mejor por combinaciones de variables que un umbral individual no captura; para los detectores no

se usan arquitecturas profundas por el volumen de datos de telemetría disponible. Las pruebas del primer interrogante se aplican por separado a cada decodificador; la degradación funcional que etiqueta las ventanas para el segundo y el tercer interrogante se define sobre el decodificador de referencia, CSP y LDA. Los tres modelos se usan con hiperparámetros fijos, sin selección interna, lo que se declara como limitación. El detector por umbrales dispara cuando alguna variable de telemetría sale del rango observado en las corridas de referencia de los otros ocho sujetos, de modo que también se evalúa dejando un sujeto fuera. Datos brutos, guiones y resultados intermedios se publican con el banco de pruebas.

Resultados

Los resultados se presentan en el orden de los seis objetivos específicos. Todas las cifras corresponden a la campaña definitiva, salvo cuando se indica la prueba piloto o la primera etapa de la campaña.

Construcción y verificación del banco de experimentación

El banco quedó construido con los seis componentes previstos y ejecutó la campaña definitiva de 855 ejecuciones sin intervención manual y sin ejecuciones fallidas, en 8,1 horas de reloj con doce ejecuciones simultáneas. En la condición de referencia, el servicio de reproducción despertó para emitir cada bloque con un error respecto del instante nominal cuyo percentil 99 fue de 1,3 microsegundos (máximo entre ejecuciones: 1,7 microsegundos), medido inmediatamente antes del envío, por lo que no incluye el tiempo de la transmisión. Ejecutar doce reproducciones simultáneas en lugar de una sola aumentó la latencia media de extremo a extremo de la referencia en 0,04 ms y la desviación del intervalo entre bloques en 0,14 ms respecto de la misma corrida ejecutada sola en la prueba piloto, sin cambiar la exactitud.

La prueba piloto, ejecutada sobre la sexta corrida de los sujetos 1, 5 y 8, confirmó que las doce condiciones de la familia uniforme produjeron efectos medibles en la telemetría y fijó la ventana móvil en ocho ensayos y el umbral de degradación en el mínimo de la referencia de cada sujeto. La primera etapa de la campaña (234 ejecuciones, corridas primera y segunda, familia uniforme) no mostró efecto funcional de ningún fallo; la campaña definitiva que se reporta a continuación incorporó la familia estructurada y las cinco corridas. La Tabla 2 presenta, para cada sujeto, la *balanced accuracy* de cada decodificador sobre la sesión de evaluación calculada fuera de línea, la obtenida en la condición de referencia del banco y el umbral resultante; los valores de CSP y LDA en el

banco coincidieron con los de fuera de línea en los nueve sujetos. La reaplicación de CSP y LDA a los segmentos guardados reprodujo la decisión tomada en línea en el 99,0 % de las 20.511 decisiones válidas de la campaña.

Tabla 2

Desempeño de referencia por sujeto: fuera de línea, en el banco sin fallo y umbral de degradación

Sujeto	CSP+LDA fuera de línea	CSP+LDA en línea (referencia)	EEGNet fuera de línea	EEGNet sobre segmentos (referencia)	Umbral (mín. de la móvil)
1	0,875	0,875	0,875	0,875	0,500
2	0,625	0,625	0,542	0,542	0,167
3	0,917	0,917	0,958	0,958	0,667
4	0,625	0,625	0,708	0,708	0,375
5	0,625	0,625	0,583	0,583	0,300
6	0,667	0,667	0,708	0,708	0,300
7	0,708	0,708	0,750	0,750	0,500
8	0,958	0,958	0,958	0,958	0,667
9	0,917	0,917	0,958	0,958	0,625

Nota. Balanced accuracy sobre los ensayos de mano izquierda y derecha de las cinco corridas de evaluación de la campaña. Fuera de línea: mediana de las cinco corridas del decodificador aplicado a las épocas. En el banco: mediana de las cinco reproducciones sin fallo (CSP+LDA en línea; EEGNet sobre los segmentos guardados). Umbral: mínimo de la balanced accuracy móvil de ocho ensayos de CSP+LDA sobre las reproducciones sin fallo del sujeto.

Efecto de los fallos sobre la integridad, la temporalidad y la disponibilidad del flujo

En la familia uniforme, cada modelo de fallo alteró la familia de variables que su definición señalaba y ninguna otra (Tabla 3). La pérdida de muestras redujo la relación entre muestras recibidas y esperadas a 0,990, 0,950 y 0,900 para las severidades de 1, 5 y 10 %, y produjo medianas de 947, 4.594 y 8.719 huecos por corrida, con aumentos de la latencia de extremo a extremo de 0,05, 0,21 y 0,45 ms, inferiores a un milisegundo, y sin cambios en la regularidad de llegada. El *jitter* no perdió muestras y elevó la desviación del intervalo entre bloques de 1,9 ms a 8,2, 30,8 y 56,3 ms, con latencias medias de 6,6,

16,8 y 34,8 ms. El retraso desplazó la latencia media a 52,7, 102,7 y 252,7 ms, valores que reproducen el desplazamiento inyectado sobre la latencia de referencia de 2,7 ms, sin alterar la integridad ni la regularidad. En las tres severidades de estas familias, las comparaciones apareadas contra la referencia resultaron significativas con un tamaño de efecto $r = 0,96$ (p corregida $\leq 0,012$).

La desconexión con cortes en instantes aleatorios afectó a la vez la integridad y la disponibilidad. Con cinco cortes por corrida, la relación de muestras recibidas fue de 0,980, 0,980 y 0,954 para cortes de 0,5, 1 y 3 s, frente a fracciones nominales de corte de 0,994, 0,987 y 0,961; la diferencia corresponde al tiempo de recuperación de la conexión. El intervalo máximo entre llegadas fue de 1.561, 1.560 y 3.598 ms para cortes de 0,5, 1 y 3 s respectivamente (Figura 6): el mismo valor para los cortes de 0,5 y 1 s, y 1.061, 560 y 598 ms por encima de la duración del corte en cada caso. Los segundos sin datos por corrida fueron 3, 3 y 13.

En la familia estructurada, la pérdida contigua dentro del ensayo redujo la relación de muestras recibidas a 0,950, 0,876 y 0,801 para el 10, 25 y 40 % de la ventana, con 48, 48 y 48 huecos por corrida, uno por ensayo, y no alteró la temporalidad. La desconexión sincronizada con el inicio de la ventana de decodificación, con un corte por ensayo, redujo la relación de muestras recibidas a 0,811, 0,811 y 0,687 para cortes de 0,5, 1 y 2 s, con 35, 35 y 83 segundos sin datos por corrida e intervalos máximos entre llegadas de 1.565, 1.565 y 2.564 ms (Figura 6); dentro de la ventana de cada ensayo quedaron disponibles, en mediana, 620, 620 y 370 de las mil muestras, de modo que la proporción de decisiones válidas fue de 1,000, 1,000 y 1,000. La Figura 7 muestra la latencia media por tipo y severidad de fallo.

Tabla 3

*Integridad, temporalidad y disponibilidad del flujo por tipo y severidad de fallo
(mediana entre sujetos)*

Tipo de fallo	Severidad	Muestras recibidas / esperadas	Huecos	Latencia media (ms)	Desv. intervalo entre bloques (ms)	Segundos sin datos
Referencia	—	1,000	0	2,7	1,9	0
Pérdida de muestras	1 %	0,990*	947*	2,7*	1,9	0
Pérdida de muestras	5 %	0,950*	4.594*	2,9*	1,9	0
Pérdida de muestras	10 %	0,900*	8.719*	3,1*	1,9	0
<i>Jitter</i>	10 ms	1,000	0	6,6*	8,2*	0
<i>Jitter</i>	50 ms	1,000	0	16,8*	30,8*	0
<i>Jitter</i>	100 ms	1,000	0	34,8*	56,3*	0
Retraso	50 ms	1,000	0	52,7*	1,9	0
Retraso	100 ms	1,000	0	102,7*	1,9	0
Retraso	250 ms	1,000	0	252,7*	1,9	0
Desconexión	0,5 s	0,980*	5*	2,7	7,2*	3*
Desconexión	1 s	0,980*	5*	2,7	7,2*	3*
Desconexión	3 s	0,954*	5*	2,7	15,2*	13*
Pérdida contigua en el ensayo	10 % de la ventana	0,950*	48*	2,8*	12,2*	0
Pérdida contigua en el ensayo	25 % de la ventana	0,876*	48*	3,0*	35,2*	0
Pérdida contigua en el ensayo	40 % de la ventana	0,801*	48*	3,0*	59,3*	29*
Desconexión en el ensayo	0,5 s	0,811*	48*	2,7	55,4*	35*
Desconexión en el ensayo	1 s	0,811*	48*	2,7	55,4*	35*
Desconexión en el ensayo	2 s	0,687*	48*	2,7	104,9*	83*

Nota. Mediana de la mediana por sujeto sobre sus cinco corridas. Huecos: discontinuidades mayores a un intervalo de muestreo en las marcas de tiempo recibidas. Latencia: llegada menos marca de tiempo nominal del último dato del bloque. El asterisco indica $p < 0,05$ en la comparación de Wilcoxon contra la referencia con corrección de Holm. Cada corrida con desconexión incluyó cinco cortes; cada corrida con desconexión en el ensayo, un corte por ensayo.

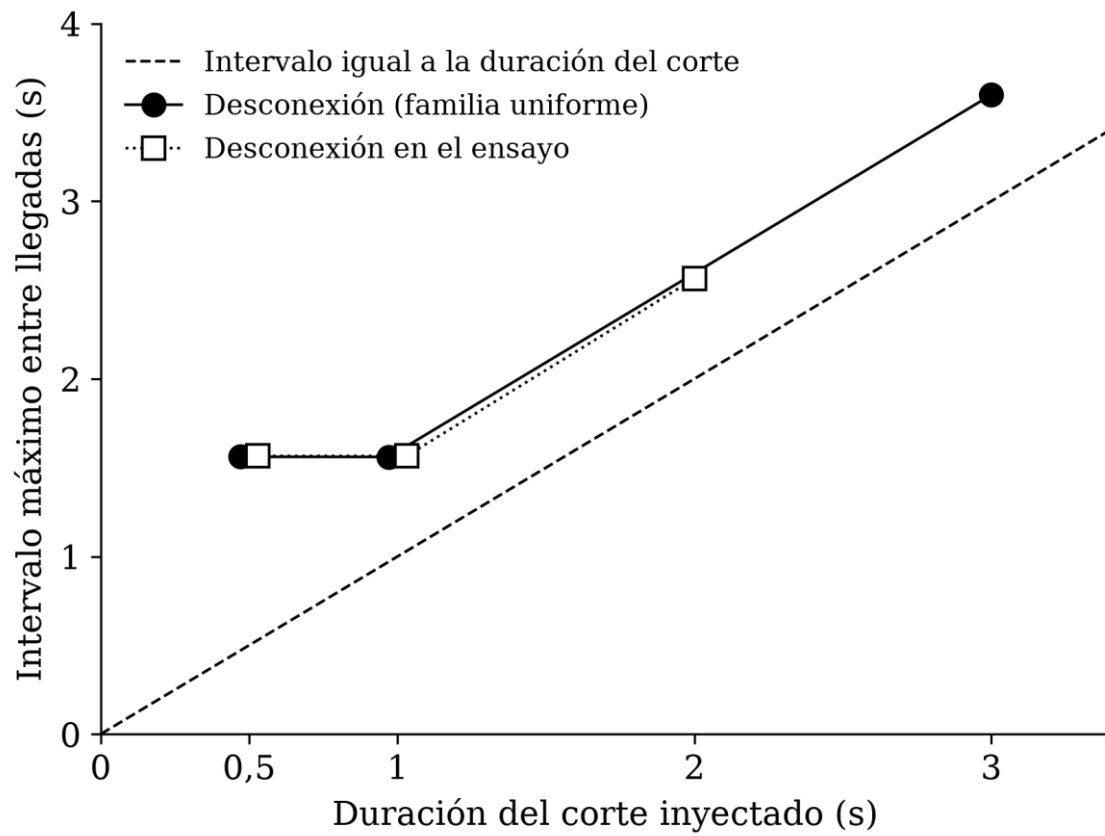


Figura 6. Intervalo máximo entre llegadas según la duración del corte inyectado, en las dos familias de desconexión. Mediana entre sujetos. La línea discontinua marca un intervalo igual a la duración del corte. Los marcadores de 0,5 y 1 s de cada familia se desplazaron levemente en horizontal para que no se superpongan.

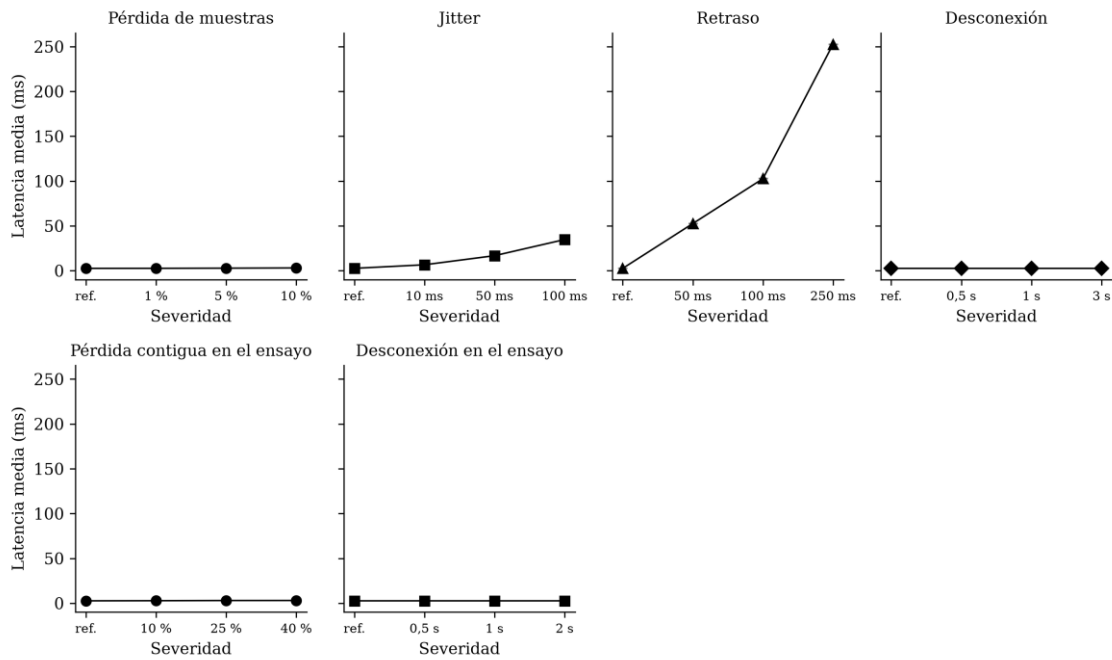


Figura 7. Latencia media de extremo a extremo por tipo y severidad de fallo. Cada panel corresponde a un tipo de fallo; el primer punto de cada panel es la condición de referencia. Los marcadores indican la mediana entre sujetos y las barras de error el rango intercuartílico (percentiles 25 y 75). Severidades: pérdida en porcentaje de muestras omitidas; jitter en desviación estándar de la demora; retraso en demora constante; desconexión en duración de cada corte; pérdida contigua en porcentaje de la ventana del ensayo; desconexión en el ensayo en duración del corte.

Efecto de los fallos sobre el desempeño del decodificador

Con el decodificador de referencia, CSP y LDA, la *balanced accuracy* no cambió con ningún tipo ni severidad de la familia uniforme (Tabla 4 y Figura 8): la mediana entre sujetos fue de 0,708 en la referencia y en 12 de las 12 condiciones uniformes, y la mediana de las diferencias apareadas fue de 0,000 en todas. Para la pérdida de muestras la prueba de Friedman no detectó diferencias ($\chi^2(3) = 0,81$, $p = 0,846$); para la desconexión la prueba de Friedman detectó diferencias globales entre condiciones ($\chi^2(3) = 8,25$, $p = 0,041$) sin que ninguna severidad difiriera de la referencia tras la corrección de Holm; para el *jitter* y el retraso no fue calculable porque las nueve observaciones por sujeto coincidieron exactamente con las de referencia. La tasa de decisiones válidas fue de 1,000

en todas las condiciones uniformes y la confianza media del clasificador se mantuvo entre 0,84 y 0,88.

El efecto funcional de los fallos temporales se observó en el retardo de decisión, no en la exactitud (Figura 9): el pipeline emitió cada decisión 1.032 ms después del fin nominal del ensayo en la referencia, 1.082, 1.131 y 1.282 ms con retrasos de 50, 100 y 250 ms, y 1.034, 1.047 y 1.077 ms con *jitter* de 10, 50 y 100 ms; la pérdida de muestras y la desconexión no modificaron ese retardo (entre 1.031 y 1.032 ms).

En la familia estructurada, la *balanced accuracy* de CSP y LDA fue de 0,708, 0,750 y 0,667 con pérdida contigua del 10, 25 y 40 % de la ventana (p corregidas de 0,375, 0,250 y 0,012; Friedman: $\chi^2(3) = 14,09$, $p = 0,003$), y de 0,750, 0,750 y 0,667 con desconexión en el ensayo de 0,5, 1 y 2 s (p corregidas de 0,062, 0,062 y 0,012; Friedman: $\chi^2(3) = 16,75$, $p < 0,001$). Para CSP y LDA, resultaron significativas las comparaciones contra la referencia de la pérdida contigua del 40 % de la ventana y la desconexión en el ensayo de 2 s.

Con el segundo decodificador, EEGNet, la *balanced accuracy* de referencia fue de 0,750 y, en la familia uniforme, fue de 0,750 en las doce condiciones (Tabla 4 y Figura 10). Para EEGNet en la familia uniforme, ninguna comparación contra la referencia resultó significativa. En la familia estructurada, EEGNet obtuvo 0,750, 0,750 y 0,708 con pérdida contigua del 10, 25 y 40 % (p corregidas de 0,594, 0,594 y 0,023; Friedman: $\chi^2(3) = 12,08$, $p = 0,007$) y 0,583, 0,583 y 0,500 con desconexión en el ensayo de 0,5, 1 y 2 s (p corregidas de 0,012, 0,012 y 0,012; Friedman: $\chi^2(3) = 20,54$, $p < 0,001$). Para EEGNet en la familia estructurada, resultaron significativas las comparaciones contra la referencia de la pérdida contigua del 40 % de la ventana, la desconexión en el ensayo de 0,5 s, la desconexión en el ensayo de 1 s y la desconexión en el ensayo de 2 s.

Tabla 4*Desempeño funcional de los dos decodificadores por tipo y severidad de fallo*

Tipo de fallo	Severidad	CSP+LDA: <i>balanced accuracy</i>	CSP+LDA: <i>p</i> (Holm)	EEGNet: <i>balanced accuracy</i>	EEGNet: <i>p</i> (Holm)	Decisiones válidas
Referencia	—	0,708	—	0,750	—	1,000
Pérdida de muestras	1 %	0,708	0,750	0,750	1,000	1,000
Pérdida de muestras	5 %	0,708	1,000	0,750	1,000	1,000
Pérdida de muestras	10 %	0,708	1,000	0,750	1,000	1,000
<i>Jitter</i>	10 ms	0,708	1,000	0,750	1,000	1,000
<i>Jitter</i>	50 ms	0,708	1,000	0,750	1,000	1,000
<i>Jitter</i>	100 ms	0,708	1,000	0,750	1,000	1,000
Retraso	50 ms	0,708	1,000	0,750	1,000	1,000
Retraso	100 ms	0,708	1,000	0,750	1,000	1,000
Retraso	250 ms	0,708	1,000	0,750	1,000	1,000
Desconexión	0,5 s	0,708	1,000	0,750	1,000	1,000
Desconexión	1 s	0,708	1,000	0,750	1,000	1,000
Desconexión	3 s	0,708	0,375	0,750	1,000	1,000
Pérdida contigua en el ensayo	10 % de la ventana	0,708	0,375	0,750	0,594	1,000
Pérdida contigua en el ensayo	25 % de la ventana	0,750	0,250	0,750	0,594	1,000
Pérdida contigua en el ensayo	40 % de la ventana	0,667	0,012*	0,708	0,023*	1,000
Desconexión en el ensayo	0,5 s	0,750	0,062	0,583	0,012*	1,000
Desconexión en el ensayo	1 s	0,750	0,062	0,583	0,012*	1,000
Desconexión en el ensayo	2 s	0,667	0,012*	0,500	0,012*	1,000

Nota. Mediana entre sujetos de la mediana por sujeto sobre sus cinco corridas. p: comparación de Wilcoxon contra la referencia con corrección de Holm dentro de cada tipo de fallo; el asterisco indica $p < 0,05$. Decisiones válidas: proporción de ensayos con decisión, según la regla de medio segundo de señal en la ventana, para el pipeline en línea. Un guion indica que la prueba no fue calculable por ausencia de variación.

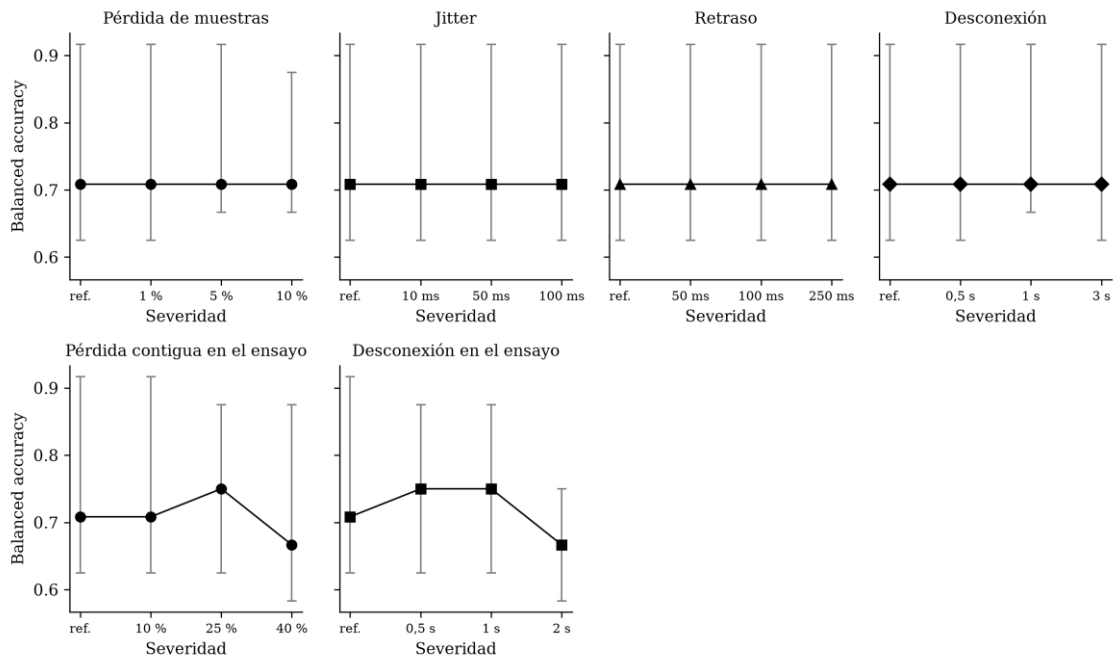


Figura 8. Balanced accuracy del decodificador de referencia (CSP y LDA) por tipo y severidad de fallo. Cada panel corresponde a un tipo de fallo; el primer punto es la referencia. Marcadores: mediana entre sujetos; barras de error: rango intercuartílico.

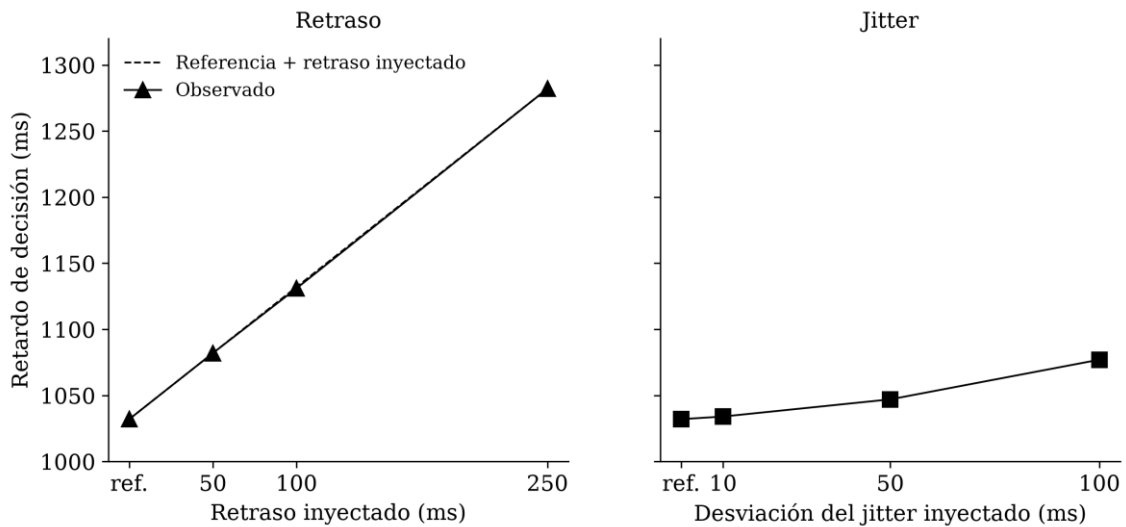


Figura 9. Retardo de decisión según el retraso y el jitter inyectados. Mediana entre sujetos del tiempo entre el fin nominal del ensayo y la emisión de la decisión; ref.: condición de referencia. En el panel del retraso, la línea discontinua marca la referencia más el retraso inyectado.

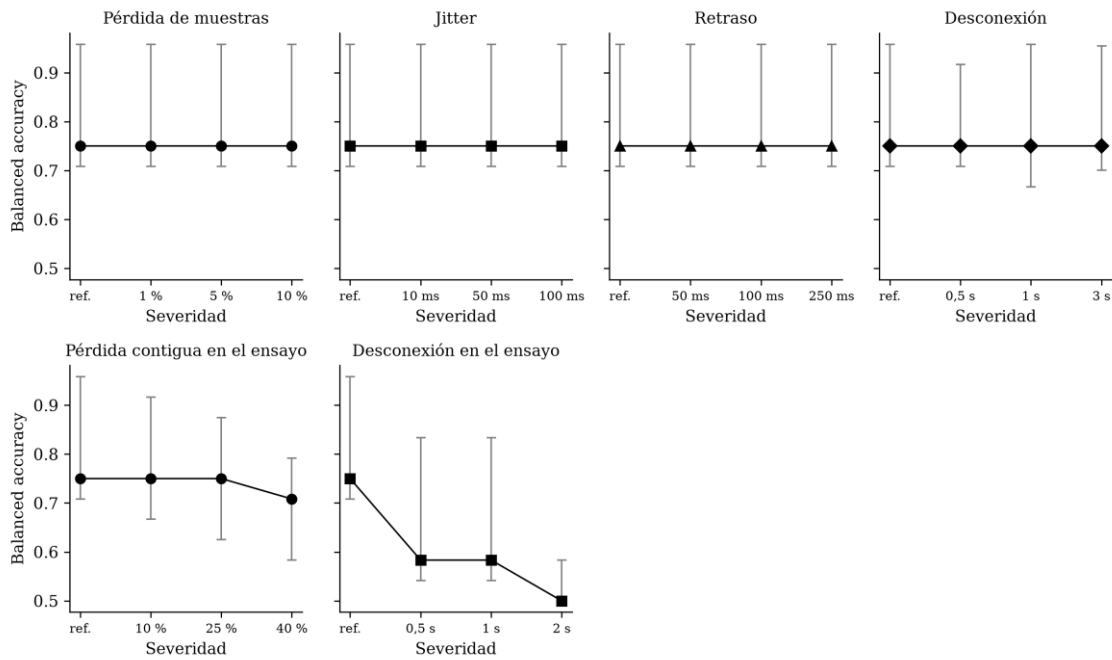


Figura 10. Balanced accuracy del segundo decodificador (EEGNet) por tipo y severidad de fallo. Evaluado sobre los segmentos guardados por el pipeline. Cada panel corresponde a un tipo de fallo; el primer punto es la referencia. Marcadores: mediana entre sujetos; barras de error: rango intercuartílico.

Divergencia entre el estado operativo de la infraestructura y el desempeño funcional

Con la ventana de ocho ensayos y el umbral por sujeto, la proporción máxima de ventanas de un segundo en que la infraestructura se mostró operativa mientras el decodificador de referencia estaba degradado fue de 0,000, considerando todas las condiciones (Tabla 5 y Figura 11). En la familia uniforme, la divergencia observada fue de signo opuesto y se concentró en la desconexión: ventanas en que el flujo no estaba operativo, por ausencia de muestras, sin que el desempeño funcional cruzara el umbral, en una proporción de 0,007, 0,007 y 0,035 para cortes de 0,5, 1 y 3 s; en la pérdida de muestras, el *jitter* y el retraso, ambas proporciones fueron nulas. En la familia estructurada, la proporción de ventanas operativas y degradadas fue de 0,000, 0,000 y 0,000 con pérdida contigua del 10, 25 y 40 %, y de 0,000, 0,000 y 0,000 con desconexión en el ensayo de 0,5, 1 y 2 s, condición en la que las ventanas no operativas y no degradadas

alcanzaron 0,085, 0,085 y 0,206. En total se registraron 77 episodios de degradación funcional en las 855 ejecuciones.

Tabla 5

Proporción de ventanas divergentes entre estado operativo y degradación funcional por condición

Tipo de fallo		Severidad	Operativo y degradado	No operativo y no degradado	Total divergentes
Referencia		—	0,000	0,000	0,000
Pérdida de muestras		1 %	0,000	0,000	0,000
Pérdida de muestras		5 %	0,000	0,000	0,000
Pérdida de muestras		10 %	0,000	0,000	0,000
<i>Jitter</i>	10 ms		0,000	0,000	0,000
<i>Jitter</i>	50 ms		0,000	0,000	0,000
<i>Jitter</i>	100 ms		0,000	0,000	0,000
Retraso	50 ms		0,000	0,000	0,000
Retraso	100 ms		0,000	0,000	0,000
Retraso	250 ms		0,000	0,000	0,000
Desconexión	0,5 s		0,000	0,007	0,007
Desconexión	1 s		0,000	0,007	0,007
Desconexión	3 s		0,000	0,035	0,035
Pérdida contigua en el ensayo	10 % de la ventana		0,000	0,000	0,000
Pérdida contigua en el ensayo	25 % de la ventana		0,000	0,000	0,000
Pérdida contigua en el ensayo	40 % de la ventana		0,000	0,073	0,079
Desconexión en el ensayo	0,5 s		0,000	0,085	0,086

Tipo de fallo	Severidad	Operativo y degradado	No operativo y no degradado	Total divergentes
Desconexión 1 s en el ensayo		0,000	0,085	0,086
Desconexión 2 s en el ensayo		0,000	0,206	0,209

Nota. Ventanas de un segundo. Operativo: llegaron muestras y no hubo excepciones en el segundo. Degradado: la última balanced accuracy móvil disponible del decodificador de referencia fue inferior al umbral del sujeto. Valores: mediana entre sujetos de la proporción por ejecución. Las medianas de cada categoría y del total se calculan por separado, por lo que las dos primeras columnas pueden no sumar exactamente el total.

Función del decodificador	Estado de la infraestructura	
	Operativa	No operativa
No degradada	Concordancia resto de las ventanas	Divergencia: interrupción hasta 0,206 según la condición
Degradada	Divergencia: falla silenciosa 0,000 en las 18 condiciones	Concordancia no se cuantificó por separado

Figura 11. Divergencia observada sobre las combinaciones de la Figura 3. Proporción de ventanas de un segundo, mediana entre sujetos de la proporción por ejecución (Tabla 5). La combinación de infraestructura no operativa y función degradada no se cuantificó por separado.

Detección de la degradación funcional a partir de la telemetría

La comparación entre el detector por umbrales y los modelos de clasificación se realizó sobre 242.250 ventanas etiquetadas, de las cuales 3.255 correspondieron a degradación funcional, agrupadas en los 77 episodios mencionados (Tabla 6). El detector por umbrales alcanzó una precisión de 0,010, una sensibilidad de 0,425 y una tasa de

falsos positivos de 0,587; la regresión logística, 0,023, 0,950 y 0,544; *random forest*, 0,036, 0,148 y 0,054; y *gradient boosting*, 0,000, 0,000 y 0,000. El mayor puntaje F1 correspondió a *random forest* (0,058). Los retardos de detección medianos fueron de 4,0, 0,0, 5,0 y — s respectivamente; un retardo nulo indica que la alarma ya estaba activa en el primer segundo degradado del episodio. La prevalencia de la degradación fue de 0,0134, valor que corresponde a la precisión de un detector que disparara al azar; la precisión de los detectores equivalió a 0,7, 1,7, 2,7 y 0,0 veces ese valor, y su precisión promedio fue de 0,012, 0,032, 0,024 y 0,029, respectivamente.

Tabla 6

Desempeño de los detectores de degradación funcional con validación dejando un sujeto fuera

Detector	Precisión	Sensibilidad	F1	Tasa de falsos positivos	Precisión promedio	Retardo de detección (s)	Episodios detectados
Umbrales	0,010	0,425	0,019	0,587	0,012	4,0	77/77
Regresión logística	0,023	0,950	0,045	0,544	0,032	0,0	74/77
<i>Random forest</i>	0,036	0,148	0,058	0,054	0,024	5,0	66/77
<i>Gradient boosting</i>	0,000	0,000	0,000	0,000	0,029	—	0/77

Nota. Umbrales: alarma cuando alguna variable de telemetría sale del rango observado en las corridas de referencia de los otros ocho sujetos. Modelos entrenados con ocho sujetos y evaluados en el restante, rotando. Retardo de detección: mediana, en segundos, entre el primer segundo degradado de cada episodio y la primera alarma dentro del episodio; un guion indica que no se detectó ningún episodio. Precisión promedio: área bajo la curva de precisión y sensibilidad, calculada a partir de la probabilidad estimada por cada modelo o, para el detector por umbrales, de la cantidad de variables fuera de rango.

Documentación y replicabilidad

El banco, los guiones de análisis, las decisiones por ensayo y la telemetría por segundo de todas las ejecuciones se publicaron con licencia MIT en el repositorio público <https://github.com/agustama27/bci-fault-bench> (versión 2.0, 27 de septiembre de 2026), junto con las versiones de software utilizadas (Python 3.12, MNE-Python 1.13.2, MNE-

LSL 1.14.0, MOABB 1.7.2, scikit-learn 1.9.1, PyTorch 2.14) y la especificación de la máquina (Amazon EC2 c6i.2xlarge, Ubuntu 24.04). Cada ejecución quedó identificada por sujeto, corrida y condición, con su semilla, de modo que la realización del fallo es repetible; por cada una se conservaron el registro del inyector, las decisiones por ensayo, la telemetría por segundo y los segmentos de señal recibidos. El ejecutor no necesitó reanudar la campaña.

Referencias

- Ali, Y. H., Bodkin, K., Rigotti-Thompson, M., Patel, K., Card, N. S., Bhaduri, B., Nason-Tomaszewski, S. R., Mifsud, D. M., Hou, X., Nicolas, C., Allcroft, S., Hochberg, L. R., Au Yong, N., Stavisky, S. D., Miller, L. E., Brandman, D. M., y Pandarinath, C. (2024). BRAND: A platform for closed-loop experiments with deep network models. *Journal of Neural Engineering*, 21(2), 026046.
<https://doi.org/10.1088/1741-2552/ad3b3a>
- Avizienis, A., Laprie, J.-C., Randell, B., y Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1), 11-33.
<https://doi.org/10.1109/TDSC.2004.2>
- Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., y Rosenthal, C. (2016). Chaos engineering. *IEEE Software*, 33(3), 35-41.
<https://doi.org/10.1109/MS.2016.60>
- Blankertz, B., Tomioka, R., Lemm, S., Kawanabe, M., y Müller, K.-R. (2008). Optimizing spatial filters for robust EEG single-trial analysis. *IEEE Signal Processing Magazine*, 25(1), 41-56. <https://doi.org/10.1109/MSP.2008.4408441>
- Card, N. S., Wairagkar, M., Iacobacci, C., Hou, X., Singer-Clark, T., Willett, F. R., Kunz, E. M., Fan, C., Vahdati Nia, M., Deo, D. R., Srinivasan, A., Choi, E. Y., Glasser, M. F., Hochberg, L. R., Henderson, J. M., Shahlaie, K., Stavisky, S. D., y Brandman, D. M. (2024). An accurate and rapidly calibrating speech neuroprosthesis. *New England Journal of Medicine*, 391(7), 609-618.
<https://doi.org/10.1056/NEJMoa2314132>

- Clisson, P., Bertrand-Lalo, R., Congedo, M., Victor-Thomas, G., y Chatel-Goldman, J. (2019). Timeflux: An open-source framework for the acquisition and near real-time processing of signal streams. En *Proceedings of the 8th Graz Brain-Computer Interface Conference 2019*. Verlag der Technischen Universität Graz.
- Gemborn Nilsson, M., Tufvesson, P., Heskebeck, F., y Johansson, M. (2023). An open-source human-in-the-loop BCI research framework: Method and design. *Frontiers in Human Neuroscience*, 17, 1129362.
<https://doi.org/10.3389/fnhum.2023.1129362>
- Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., Parkkonen, L., y Hämäläinen, M. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7, 267.
<https://doi.org/10.3389/fnins.2013.00267>
- Hernández Sampieri, R., Fernández Collado, C., y Baptista Lucio, M. del P. (2014). *Metodología de la investigación* (6.^a ed.). McGraw-Hill.
- Jayaram, V., y Barachant, A. (2018). MOABB: Trustworthy algorithm benchmarking for BCIs. *Journal of Neural Engineering*, 15(6), 066011.
<https://doi.org/10.1088/1741-2552/aadea0>
- Karimov, J., Rabl, T., Katsifodimos, A., Samarev, R., Heiskanen, H., y Markl, V. (2018). Benchmarking distributed stream data processing systems. En *2018 IEEE 34th International Conference on Data Engineering (ICDE)* (pp. 1507-1518). IEEE. <https://doi.org/10.1109/ICDE.2018.00169>
- Kothe, C., Shirazi, S. Y., Stenner, T., Medine, D., Boulay, C., Grivich, M. I., Artoni, F., Mullen, T., Delorme, A., y Makeig, S. (2025). The lab streaming layer for

synchronized multimodal recording. *Imaging Neuroscience*, 3.

<https://doi.org/10.1162/IMAG.a.136>

Lab Streaming Layer. (s. f.-a). *Frequently asked questions*. Lab Streaming Layer documentation. <https://labstreaminglayer.readthedocs.io/info/faqs.html>

Lab Streaming Layer. (s. f.-b). *Supported devices and tools*. Lab Streaming Layer documentation.

https://labstreaminglayer.readthedocs.io/info/supported_devices.html

Lawhern, V. J., Solon, A. J., Waytowich, N. R., Gordon, S. M., Hung, C. P., y Lance, B.

J. (2018). EEGNet: A compact convolutional neural network for EEG-based brain-computer interfaces. *Journal of Neural Engineering*, 15(5), 056013.

<https://doi.org/10.1088/1741-2552/aace8c>

Lotte, F., Bougrain, L., Cichocki, A., Clerc, M., Congedo, M., Rakotomamonjy, A., y

Yger, F. (2018). A review of classification algorithms for EEG-based brain-computer interfaces: A 10 year update. *Journal of Neural Engineering*, 15(3),

031005. <https://doi.org/10.1088/1741-2552/aab2f2>

Memmott, T., Koçanaoğulları, A., Lawhead, M., Klee, D., Dudy, S., Fried-Oken, M., y

Oken, B. (2021). BciPy: Brain-computer interface software in Python. *Brain-Computer Interfaces*, 8(4), 137-153.

<https://doi.org/10.1080/2326263X.2021.1878727>

Meng, J., Edelman, B. J., Olsoe, J., Jacobs, G., Zhang, S., Beyko, A., y He, B. (2018). A

study of the effects of electrode number and decoding algorithm on online EEG-based BCI behavioral performance. *Frontiers in Neuroscience*, 12, 227.

<https://doi.org/10.3389/fnins.2018.00227>

- Metzger, S. L., Littlejohn, K. T., Silva, A. B., Moses, D. A., Seaton, M. P., Wang, R., Dougherty, M. E., Liu, J. R., Wu, P., Berger, M. A., Zhuravleva, I., Tu-Chan, A., Ganguly, K., Anumanchipalli, G. K., y Chang, E. F. (2023). A high-performance neuroprosthesis for speech decoding and avatar control. *Nature*, 620(7976), 1037-1046. <https://doi.org/10.1038/s41586-023-06443-4>
- Natella, R., Cotroneo, D., y Madeira, H. S. (2016). Assessing dependability with software fault injection: A survey. *ACM Computing Surveys*, 48(3), 1-55. <https://doi.org/10.1145/2841425>
- Nicolas-Alonso, L. F., y Gomez-Gil, J. (2012). Brain computer interfaces, a review. *Sensors*, 12(2), 1211-1279. <https://doi.org/10.3390/s120201211>
- Renard, Y., Lotte, F., Gibert, G., Congedo, M., Maby, E., Delannoy, V., Bertrand, O., y Lécuyer, A. (2010). OpenViBE: An open-source software platform to design, test, and use brain-computer interfaces in real and virtual environments. *Presence: Teleoperators and Virtual Environments*, 19(1), 35-53. <https://doi.org/10.1162/pres.19.1.35>
- Roy, Y., Banville, H., Albuquerque, I., Gramfort, A., Falk, T. H., y Faubert, J. (2019). Deep learning-based electroencephalography analysis: A systematic review. *Journal of Neural Engineering*, 16(5), 051001. <https://doi.org/10.1088/1741-2552/ab260c>
- Santamaría-Vázquez, E., Martínez-Cagigal, V., Marcos-Martínez, D., Rodríguez-González, V., Pérez-Velasco, S., Moreno-Calderón, S., y Hornero, R. (2023). MEDUSA©: A novel Python-based software ecosystem to accelerate brain-computer interface and cognitive neuroscience research. *Computer Methods and*

Programs in Biomedicine, 230, 107357.

<https://doi.org/10.1016/j.cmpb.2023.107357>

Schalk, G., McFarland, D. J., Hinterberger, T., Birbaumer, N., y Wolpaw, J. R. (2004).

BCI2000: A general-purpose brain-computer interface (BCI) system. *IEEE*

Transactions on Biomedical Engineering, 51(6), 1034-1043.

<https://doi.org/10.1109/TBME.2004.827072>

Tangemann, M., Müller, K.-R., Aertsen, A., Birbaumer, N., Braun, C., Brunner, C.,

Leeb, R., Mehring, C., Miller, K. J., Müller-Putz, G. R., Nolte, G., Pfurtscheller,

G., Preissl, H., Schalk, G., Schlögl, A., Vidaurre, C., Waldert, S., y Blankertz, B.

(2012). Review of the BCI Competition IV. *Frontiers in Neuroscience*, 6, 55.

<https://doi.org/10.3389/fnins.2012.00055>

Urigüen, J. A., y Garcia-Zapirain, B. (2015). EEG artifact removal—state-of-the-art and

guidelines. *Journal of Neural Engineering*, 12(3), 031001.

<https://doi.org/10.1088/1741-2560/12/3/031001>

Vogel, A., Henning, S., Perez-Wohlfeil, E., Ertl, O., y Rabiser, R. (2024). A

comprehensive benchmarking analysis of fault recovery in stream processing

frameworks. En *Proceedings of the 18th ACM International Conference on*

Distributed and Event-Based Systems (DEBS '24) (pp. 171-182). ACM.

<https://doi.org/10.1145/3629104.3666040>

Willett, F. R., Kunz, E. M., Fan, C., Avansino, D. T., Wilson, G. H., Choi, E. Y.,

Kamdar, F., Glasser, M. F., Hochberg, L. R., Druckmann, S., Shenoy, K. V., y

Henderson, J. M. (2023). A high-performance speech neuroprosthesis. *Nature*,

620(7976), 1031-1036. <https://doi.org/10.1038/s41586-023-06377-x>

- Wilson, J. A., Mellinger, J., Schalk, G., y Williams, J. (2010). A procedure for measuring latencies in brain-computer interfaces. *IEEE Transactions on Biomedical Engineering*, 57(7), 1785-1797.
<https://doi.org/10.1109/TBME.2010.2047259>
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., y Wesslén, A. (2012). *Experimentation in software engineering*. Springer. <https://doi.org/10.1007/978-3-642-29044-2>
- Wolpaw, J. R., Birbaumer, N., McFarland, D. J., Pfurtscheller, G., y Vaughan, T. M. (2002). Brain-computer interfaces for communication and control. *Clinical Neurophysiology*, 113(6), 767-791. [https://doi.org/10.1016/S1388-2457\(02\)00057-3](https://doi.org/10.1016/S1388-2457(02)00057-3)
- Zheng, J., Li, Y., Chen, L., Wang, F., Gu, B., Sun, Q., Gao, X., y Zhou, F. (2025). Effects of packet loss on neural decoding effectiveness in wireless transmission. *Brain Sciences*, 15(3), 221. <https://doi.org/10.3390/brainsci15030221>